Week 4: More ML and Model Optimization

MAISI

Building Interpretable AI in Healthcare
Michigan Data Science Team - Winter 2026

Week 4 Agenda

01. Icebreaker!

Every great project work session starts off with a great icebreaker.

02. Cross-Validation & Regularization

How can we be certain that our model generalizes well to unseen data?

03. More ML Algorithms

What are some other machine learning algorithms that can be beneficial to making our predictions?

04. Probability Calibration

How can we get accurate, interpretable probabilities for classification predictions?

01 Fun Icebreaker!!

Get to know us and start to learn about each other with a fun icebreaker!

Icebreaker - Week 4

Share with the people around you!! :)

What are your thoughts on the following philosophical questions?

- If a tree falls in a forest when no one is around to hear it, does it even make a sound?
- If a book is written in a language that no one can understand, can it contain knowledge?
- If you constantly replace the parts of a ship with new parts, at what point is it no longer the same ship?
- What is the sound of one hand clapping?
- Can an idea exist if no one ever thinks it?

02 Cross Validation and Regularization

How can we be certain that our model generalizes well to unseen data, and how can we prevent overfitting?

What is Cross Validation?

- **Cross Validation** is a resampling technique used in machine learning to evaluate the performance of a machine learning model

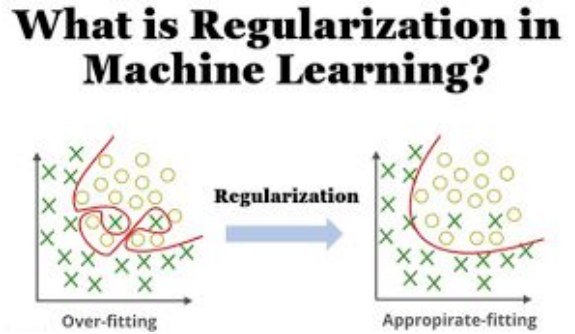
- This is achieved by splitting a dataset into *k* "folds," which are randomized and equally-sized subsets of the data, and training the model on k - 1 of them to predict the outcome in the case of the remaining fold (called the validation set)

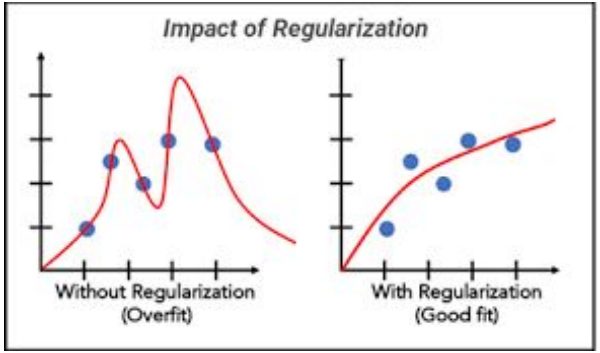
- This process is typically run *k* times, switching off which fold is the validation set each time. Model performance can then be evaluated using R^2 or (TP+TN)/(TP+TN+FP+FN)

What is Regularization?

- **Regularization** is a technique used to prevent overfitting in an ML model by penalizing complexity and large weights on particular features.
    - Overfitting: Memorizing a practice test rather than learning the concepts.

- Modifies the **loss function** of the model by adding a penalty term.
    - Loss function: mathematical model quantifying the error between predicted and actual values.



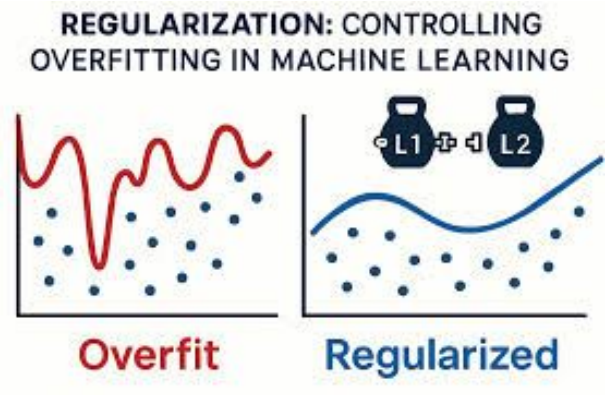


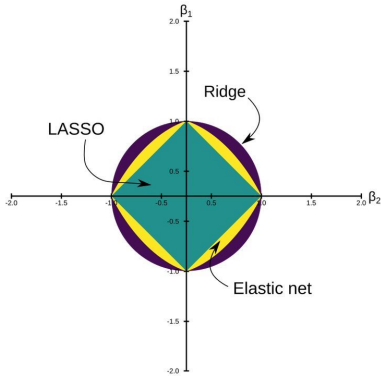
What are the Main Kinds of Regularization?

- **LI (Lasso):** Adds the absolute value of the coefficients to the cost function

- **L2 (Ridge):** Adds the squared magnitude of all of the coefficients to the cost function.

- **ElasticNet (LI + L2):** Combines both LI and L2 regularization to the "cost function" of the model. In this way, you get the "best of both worlds."





When Should I Use Each Method?

- **L1 (Lasso):** Good for selecting quality features and simplifying the coefficients of the features in your dataset.

- **L2 (Ridge):** Ideal to handle multicollinearity issues (multiple variables having a high correlation, skewing the influence of that feature on the target).

- **ElasticNet (L1 + L2):** Works for almost every case in which you would use Lasso or Ridge, though it is not advisable to use if you have an exceedingly small number of observations or if you need a very simple model that can be calculated more easily.

03 More ML Algorithms

What are a couple of other ML algorithms we can use to make predictions?

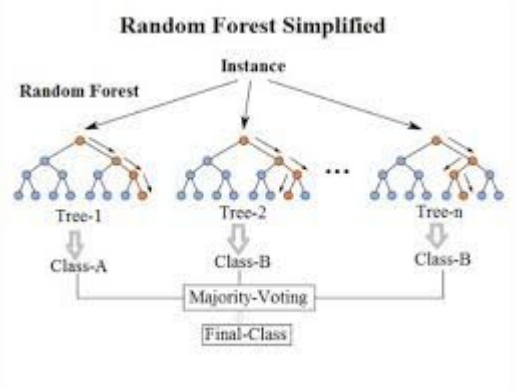
ML Algorithms - Random Forest

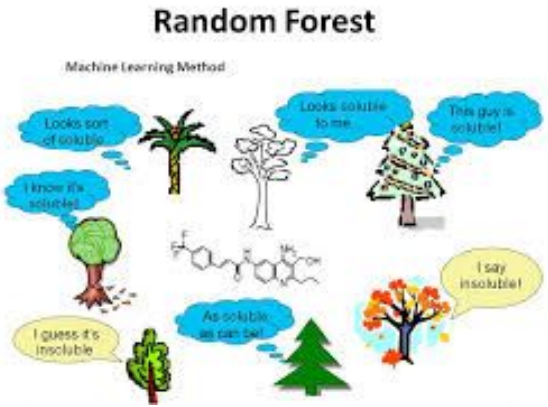
- Aggregates decisions made over multiple different tree and averages them out to find the most likely classification

- Made up of numerous different "trees" that are trained on different subsets of the data, sampled with replacement

- Works on both classification and regression tasks
  - Classification: takes the **mode** (most common)
  - Regression: takes the average (Σ/n)

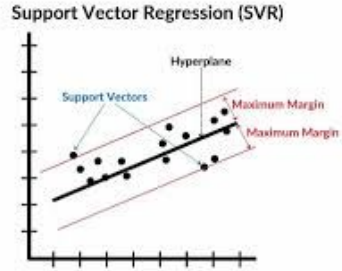
- Highly versatile and typically very accurate predictions!



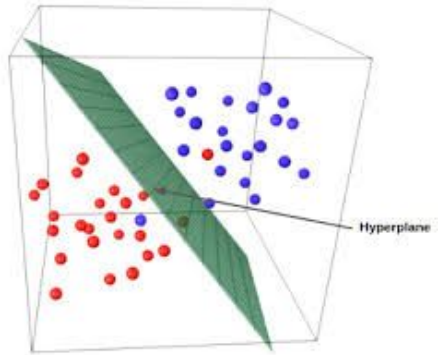


ML Algorithms - Support Vector Regressor

- **Support Vector Regressor (SVR)** is used mainly to predict continuous numerical values by attempting to fit a *hyperplane* with a defined tolerance margin around data
  - Goal: get as many data points inside the margin as possible while minimizing error from the hyperplane.



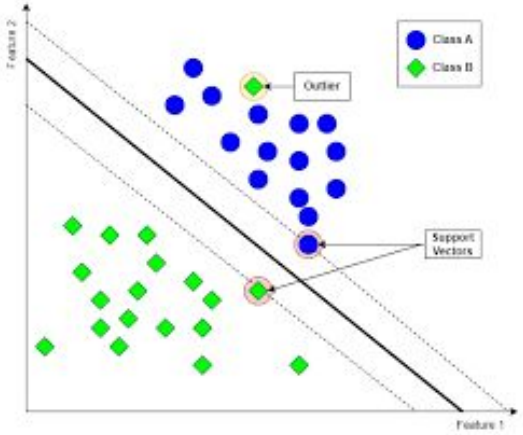
- Basically ignores points that don't fall on the hyperplane but within the tolerance margin in the error calculations.



- Often used in contexts like stock market prediction, forecasting (weather, healthcare outcomes), and other regression tasks where complex, non-linear relationships between features exist.

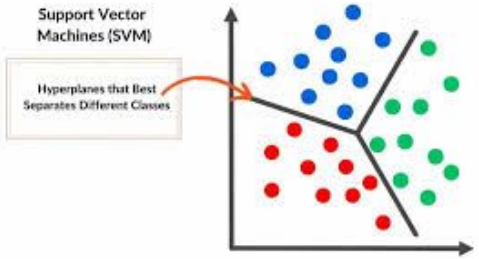
ML Algorithms - Support Vector Machine

- **Support Vector Machine (SVM)** is very similar to Support Vector Regressor, but used in the context of *classification* (which encompasses all of your projects)

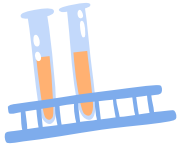


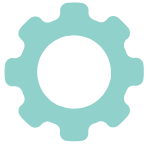
- Works by fitting hyperplane(s) that best separate classes in the dataset, often incorporating a tolerance margin like in the last problem when calculating error.

- Very effective in high-dimensional datasets and accomplishes the same goal as SVR in reducing overfitting.
    - However, SVM has the same issues of being unsuitable for data with high multicollinearity.



04





Probability Calibration

How can we get accurate, interpretable probabilities for classification predictions?

What is Probability Calibration?

- **Probability Calibration** quantifies uncertainty by helping our models interpret data rationally when absolute confidence is not possible.
  - Allows the model to spit out its "belief" in an outcome rather than a prediction with a handful of summary statistics as to how it got to that conclusion

- A lot of machine learning models, as a consequence of the origin of the field, are founded on top of core statistical principles, including quantifying uncertainty.
  - E.g. AI chatbots as complex probability models, Naive Bayes, logistic regression

- This information on how likely the model claims its prediction is to be true, we can tune parameters and adjust weights to better optimize our models to make the best conclusions possible.

Why is Probability Calibration Important?

- Important to note: the proportion of observations that fit into each classification category **do not** reflect the empirical (real-world) prevalence of these categories.
  - With *probability calibration*, we can map these values to interpretable, accurate metrics

- This step is crucial in the model optimization process to make sure that the likelihoods our model returns is accurate to the real-world reality.

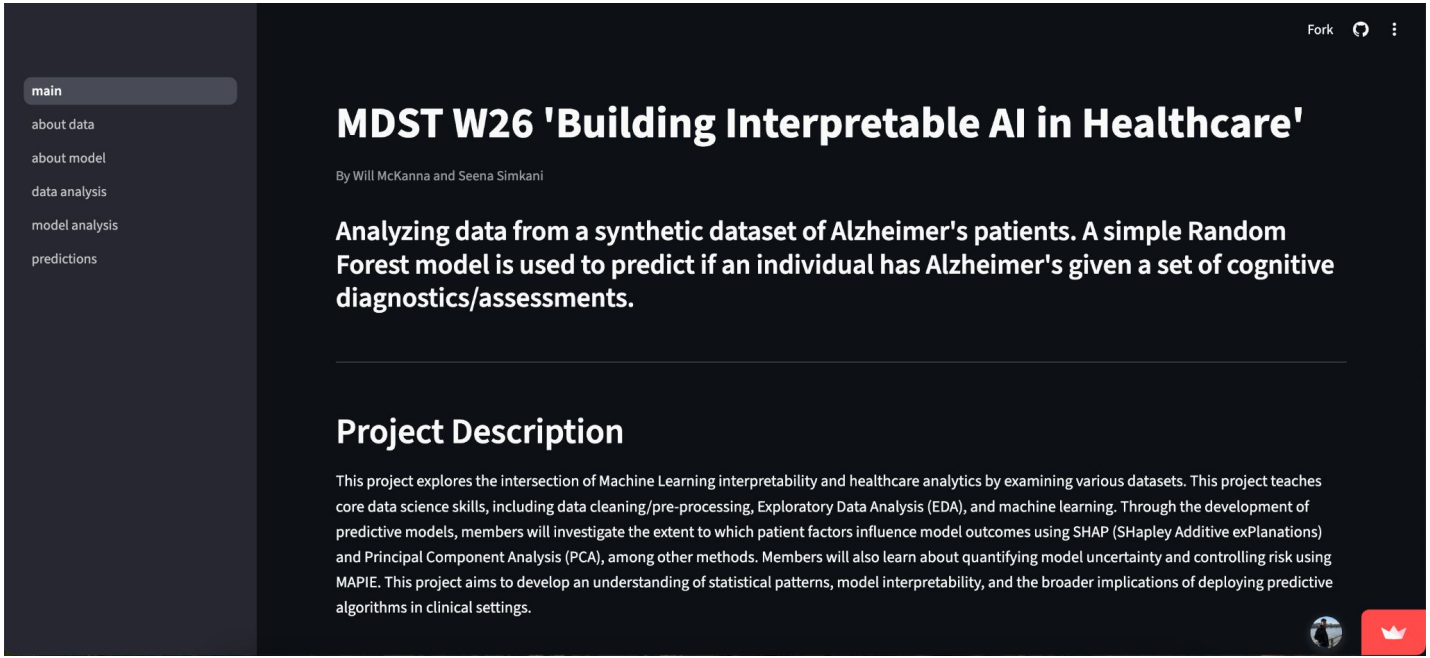
- Ex. A model predicts a category with 70% confidence, and that category has about a 70% chance of belonging to that category in reality.

- This is an improvement over straight-up binary classification, especially in a context like healthcare where the likelihood of an event is just as important as the diagnosis.

05 References

Here are a handful of useful guides to high-utility functions, Git demos, and Colab explanations!

Our Demo! (for reference)



Link:
https://mdst-ai-in-healthcare.streamlit.app/

Project Timeline

Week 1 (01/25): Icebreaker/EDA intro, choosing a good dataset

Week 2 (02/01): Data Visualization, Further EDA

Week 3 (02/08): Feature Engineering and Intro to Machine Learning

Week 4 (02/15): Machine Learning and Model Optimization

Week 5 (02/22): Conformal Predictions, Risk Control, and Multi-Class Calibration

NO MEETING 03/01 OR 03/08 – SPRING BREAK

Week 6 (03/15): SHAP Values and Feature Importance Interpretation

Week 7 (03/22): Intro to Streamlit and Project Work Time

Weeks 8 + 9 (03/29 + 04/05): Final Project work time + Data Science Night prep!

Important Functions to Know (pt. 1)

Syntax	Description
import pandas as pd	Import the **pandas** library, using the alias **pd** for convenience
import numpy as np	Import the **NumPy** library, using alias **np** for numerical operations
import seaborn as sns	Import **Seaborn** for statistical data visualization
df = pd.read_csv('file.csv')	Load a **CSV** file into a **pandas DataFrame** for data manipulation
df.head()	Returns the **first 5 rows** of the DataFrame, useful for quickly inspecting data
df.info()	Provides a concise **summary** of the DataFrame, including **data types** and **non-null values**

Important Functions to Know (pt. 2)

Syntax	Description
df.describe()	Generates **descriptive statistics** of numerical columns (mean, median, quartiles, etc.)
df[‘column_name’]	Access a **specific column** in the DataFrame, works like a key in a dictionary
df.drop(columns=[‘col’...])	**Drops specified columns** from the DataFrame,use inplace=True to modify the original DataFrame
df.index	Returns the **index labels** of the DataFrame
df.isnull()	returns a DataFrame of **boolean values**, where each entry indicates whether the corresponding value in df is NaN (missing)

Important Functions to Know (pt. 3)

Syntax	Description
df[‘column’].value_counts()	Returns the **count** of **unique values** in a specific column
df[‘column’].mean()	Returns the **mean value** of a numerical column
df.corr()	Computes **correlation** for numerical columns to understand relationships
df.columns	Lists all **column names** in the **DataFrame**, useful for renaming or viewing dataset structure
df.shape	Returns **# of rows and columns**

Important Functions to Know (pt. 4)

Syntax	Description
df.rename(columns={'old' : 'new'}, inplace=True)	Renames **specific columns** to new names
df.groupby(‘category’)[‘value’]	Groups the DataFrame using a specified column to perform **aggregate functions** (e.g., .sum(), .mean())
df[‘new_column’] = df[‘column’].apply(function)	**Applies** a function to each element or column/row
df.shape	Returns a **tuple** representing the **dimensions** (rows, columns) of the DataFrame
df.dropna()	Removes rows or columns containing **missing values**

Important Functions to Know (pt. 5)

Syntax	Description
df.to_numpy()	Converts Pandas DataFrame into a **NumPy array**
np.shape	Returns a **tuple** representing the **dimensions** (rows, columns) of the array (similar to df.shape)
np.reshape()	**Reshapes** an original numpy array to the specified dimensions (rows, columns)
np.array()	Creates an **N-dimensional array** (ndarray) which is more **memory efficient** than standard python lists
np.unique()	Returns the **unique elements** in a NumPy array (also returns the **counts of each element** given the keyword argument, return_counts=True)

Important Functions to Know (pt. 6)

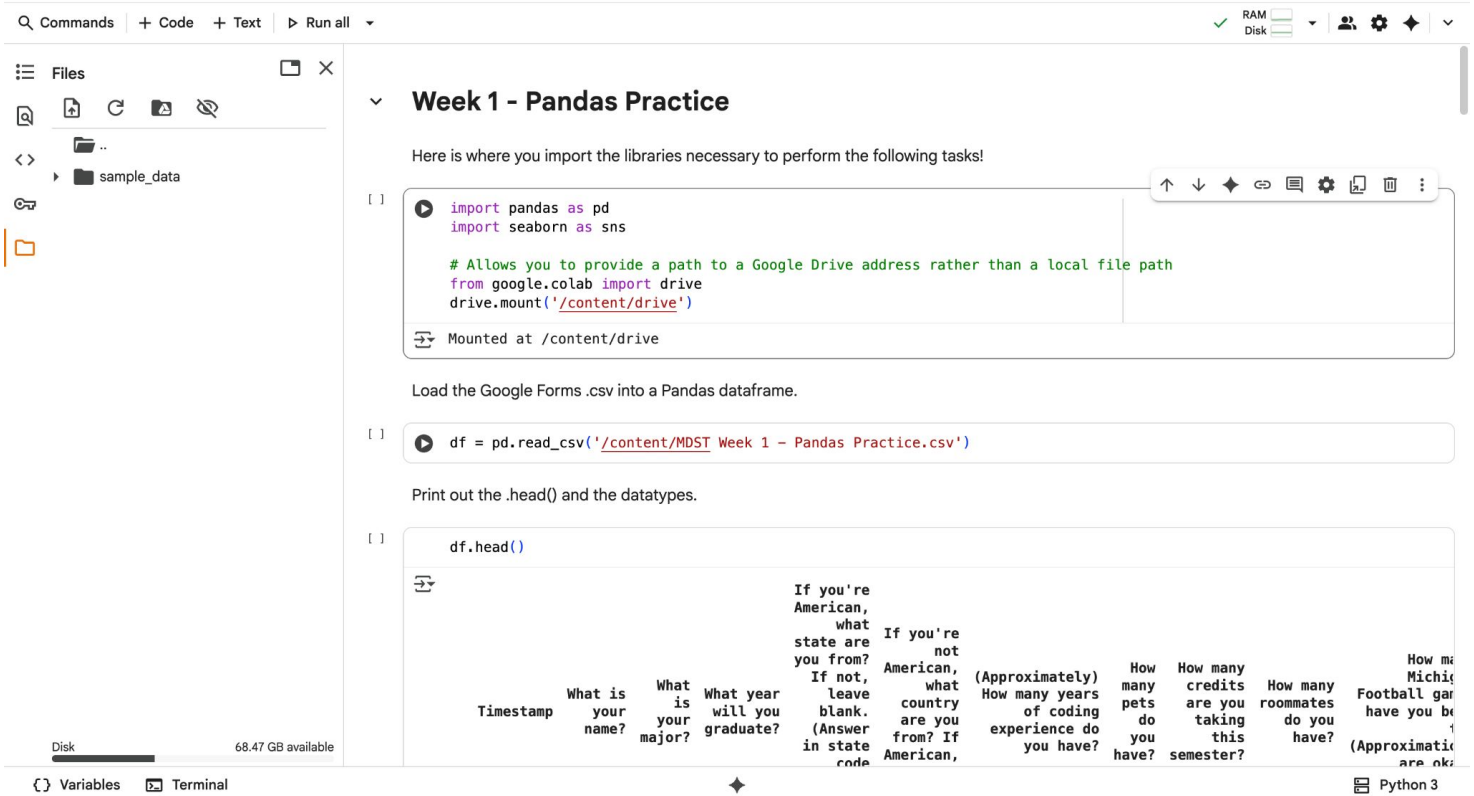
Syntax	Description
pd.cut()	Turns **continuous numerical data** into **categorical data**
pd.get_dummies(drop_first = True)	**Dummy encodes** categorical data (used with drop_first argument)
OrdinalEncoder()	Function for **encoding categorical information** that has an order
train_test_split(X, y, test_size=0.2, random_state=42)	**Splits data** into a **training set** and a **testing set**. The test set size can be changed
LinearRegression()	Used for initializing a **linear regression** model
LogisticRegression()	Used for initializing a **logistic regression** model

Important Functions to Know (pt. 7)

Syntax	Description
StandardScaler()	Turns **continuous numerical data** into **categorical data**
model.fit(X_train, y_train)	**Fits a model** to the training data
model.predict(X_test)	Uses the **test data** to make predictions
.coef_	Attribute for getting the **models coefficients** (for linear and logistic regression)
mean_squared_error()	Calculates the **average squared distance** between the predicted and actual data points
mean_absolute _error()	Calculates the **average distance** between the predicted and actual values

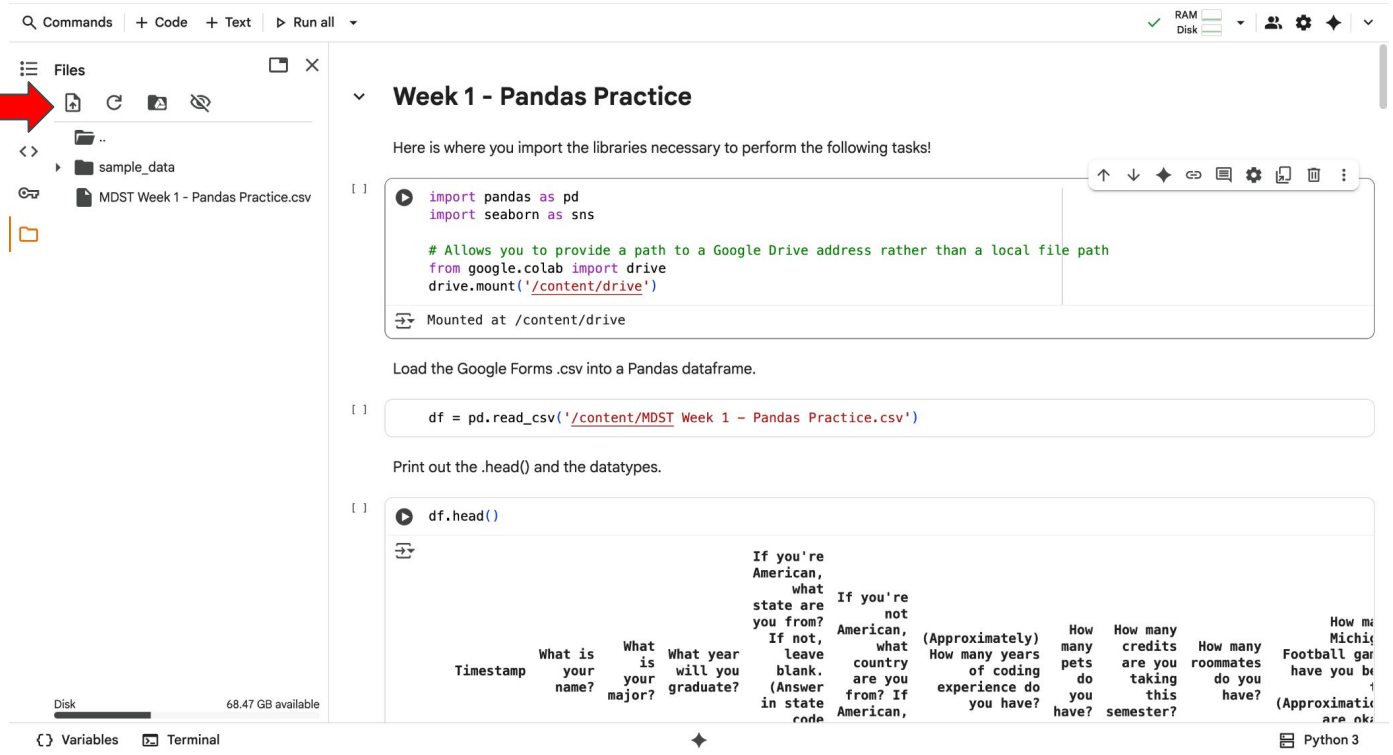
How to Upload Files into Colab

Click on the folder in the sidebar



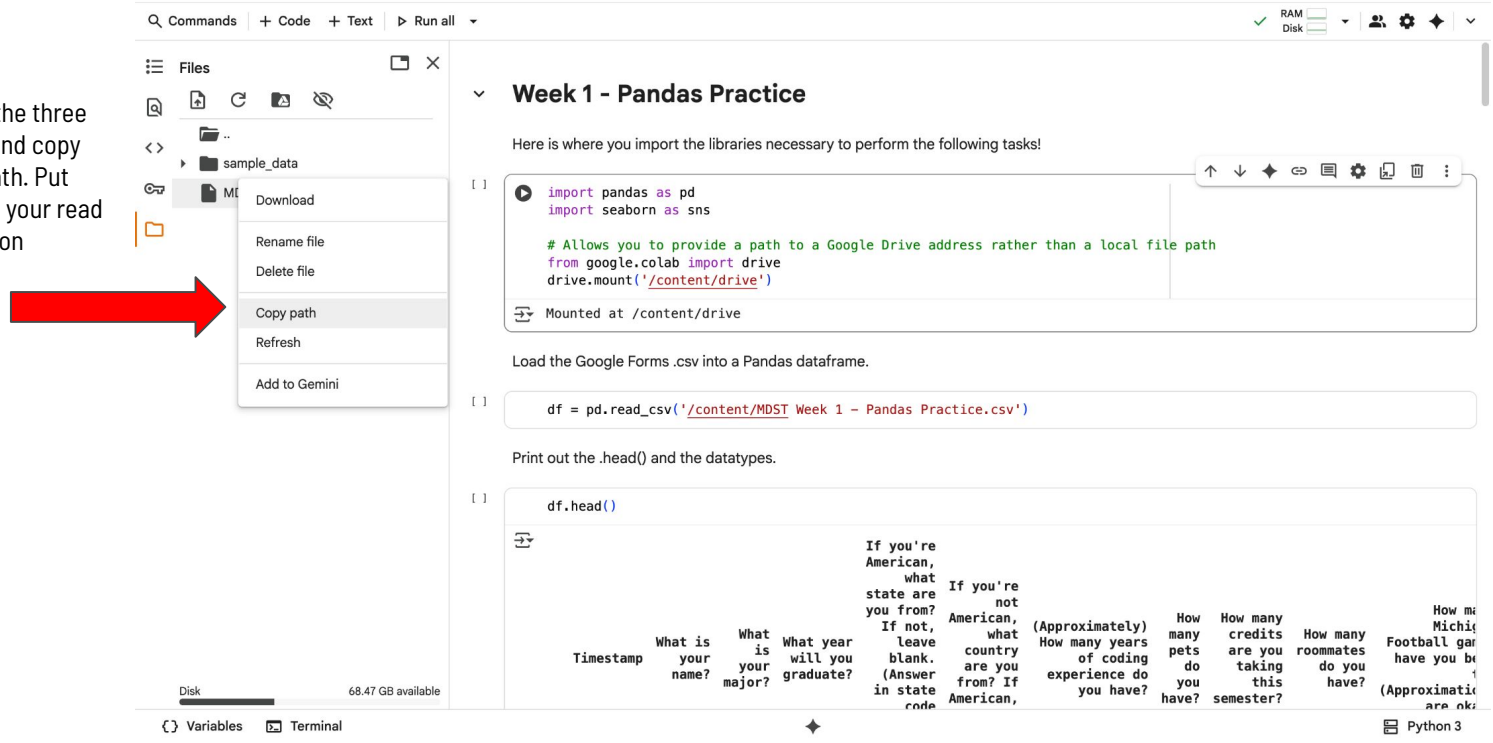
How to Upload Files into Colab

Click the upload button and select the file you want to upload



How to Upload Files into Colab

Click the three dots and copy the path. Put this in your read function



How to Mount Your Google Drive

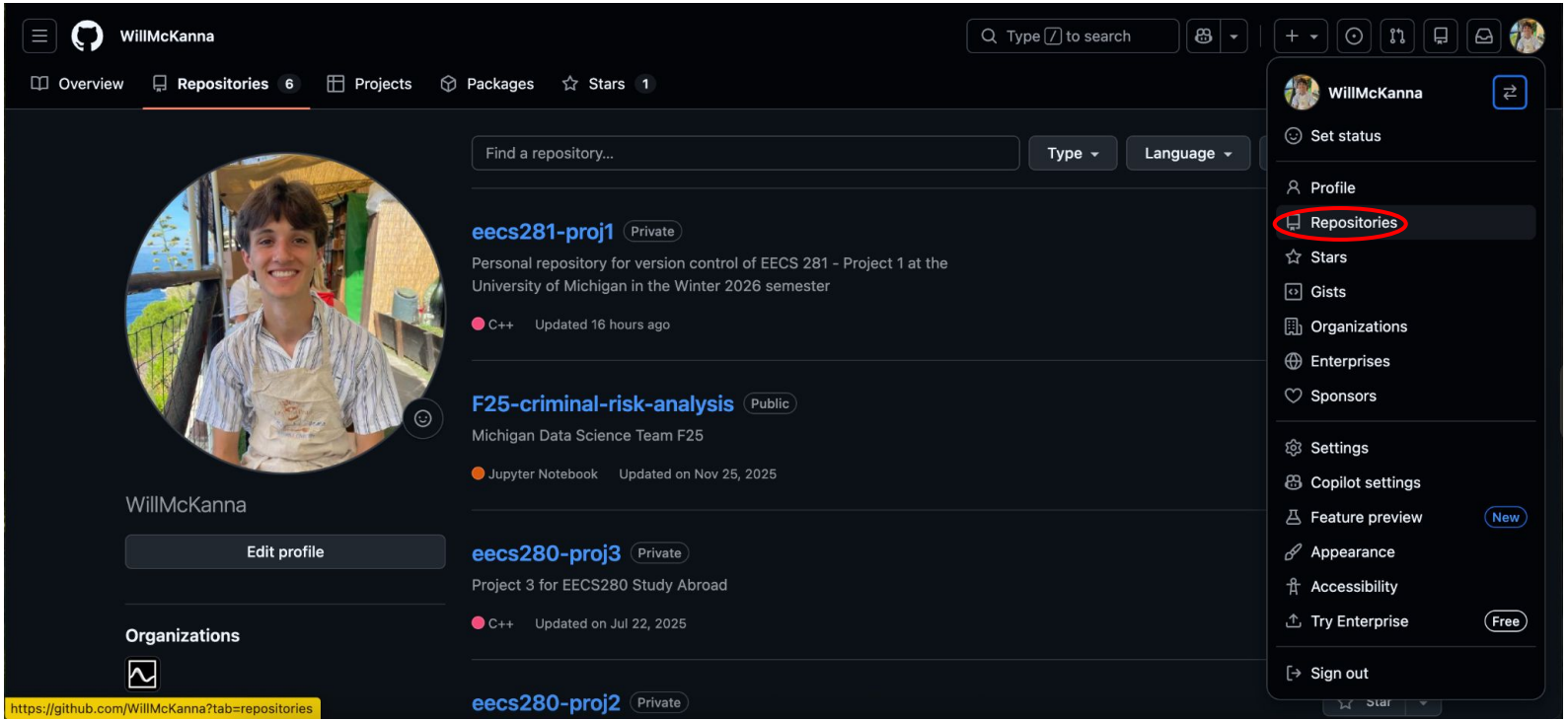
```
from google.colab import drive

drive.mount('/content/drive')


pd.read_csv('/content/drive/MyDrive/[FILE NAME]')
```

****NOTE:** If you saved your dataset in a folder (not just loose in your MyDrive folder), you will need to add further code to the file path before the file name in the final line. For example, if I saved my data called “alzheimers.csv” in a folder called “MDST-W26,” my read_csv statement would read:

```
pd.read_csv('/content/drive/MyDrive/MDST-W26/alzheimers.csv')
```

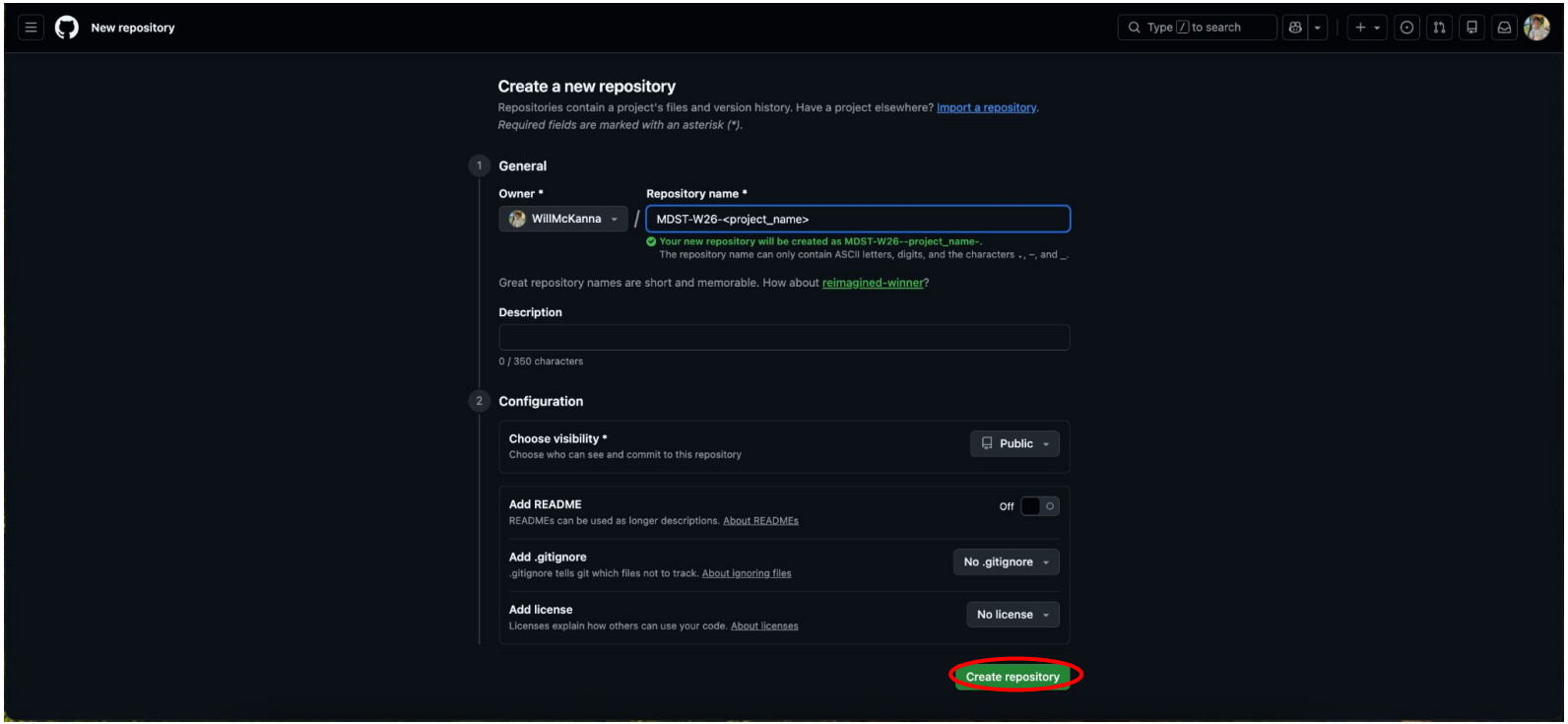
How to Create a GitHub Repository (yay for version control!)

I. Navigate to GitHub, click your profile picture, and select “Repositories.” Then, click the green “New” button next to the “Language” dropdown menu



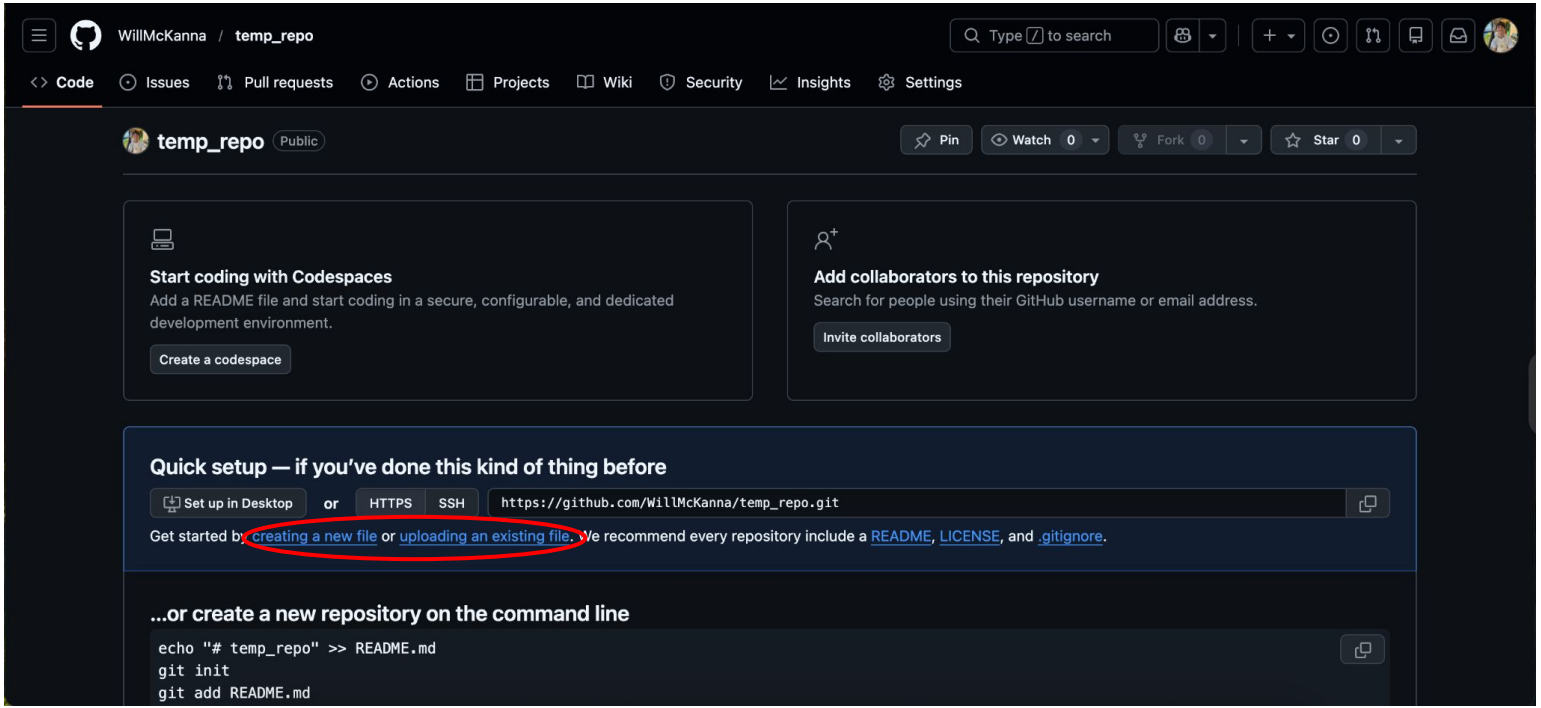
How to Create a GitHub Repository (yay for version control!)

2. Name your repository something memorable, then click “Create Repository”



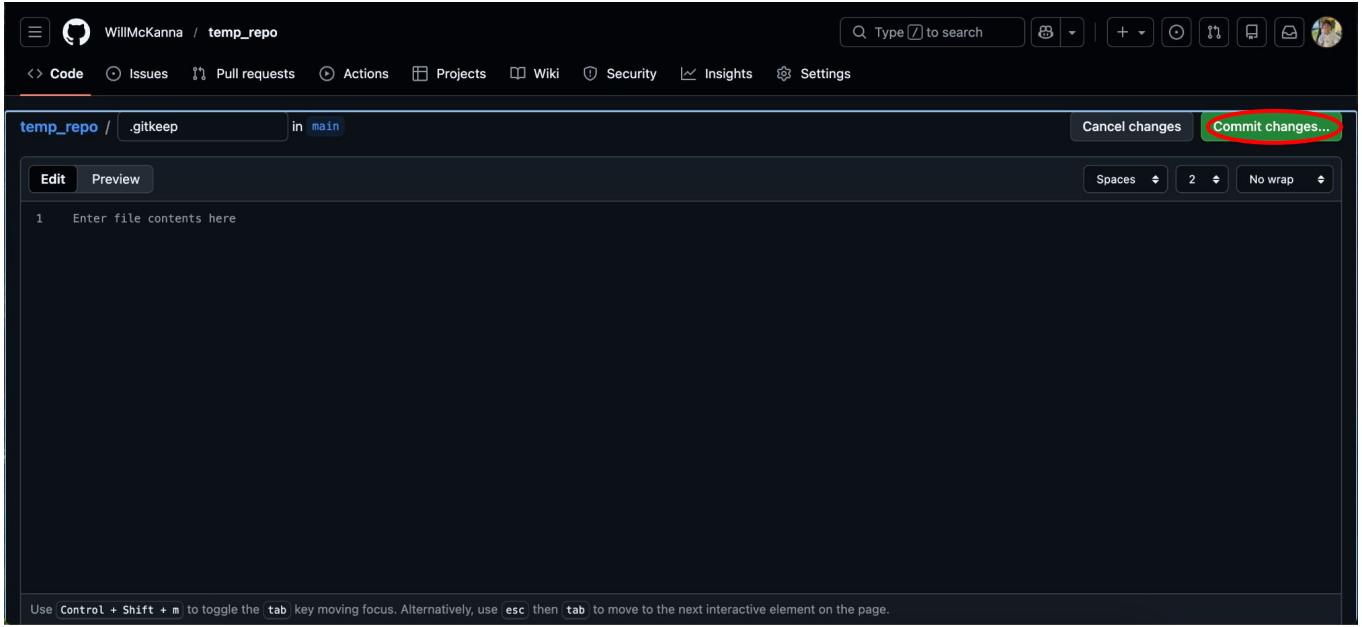
How to Create a GitHub Repository (yay for version control!)

3. Then, populate your repo either with your files, OR click “creating a new file”



How to Create a GitHub Repository (yay for version control!)

3(b). If you click “create a new file,” entitle it “.gitkeep” and click “Commit changes”



How to Create a GitHub Repository (yay for version control!)

4. Your repo is made! Continue adding to it by clicking the “Add file” dropdown menu